qual_by_position.py 사용법 및 코드 설명서

대상 파일: lab11/src/qual_by_position.py

1. 프로그램의 목적

FASTQ 파일에 들어 있는 모든 read의 품질 문자열을 Phred 점수로 바꾸고, **같은 위치(1번째 염기, 2번째 염기, ...)끼리 점수를 모아 위치별 평균 품질**을 구해 출력한다.

일루미나 read는 보통 뒤쪽으로 갈수록 품질이 떨어진다. 이 프로그램의 결과를 보면 read의 어느 위치부터 품질이 낮아지는지 알 수 있고, 트리밍(trimming) 위치를 정할 때 참고할 수 있다. FastQC의 "Per base sequence quality" 그래프와 같은 정보를 숫자로 보여 준다.

2. 실행 방법

lab11 폴더에서 다음과 같이 실행한다.

```
python3 src/qual_by_position.py data/SRR1553607_1.fastq
```

형식:

```
python3 src/qual_by_position.py <FASTQ 파일>
```

결과를 파일로 저장하려면 리다이렉션(>)을 쓴다.

```
python3 src/qual_by_position.py data/SRR1553607_1.fastq > doc/qual_by_position.tsv
```

- 인자는 sys.argv[1]로 받는 FASTQ 파일 경로 하나이다.
- 인자 개수를 검사하는 부분은 없다. 인자를 주지 않으면 IndexError가 난다.

3. 입력 형식

FASTQ 형식의 텍스트 파일이다. read 하나는 **4줄**로 이루어진다.

줄	내용	예 (data/SRR1553607_1.fastq의 첫 read)
1	@로 시작하는 read 이름	@SRR1553607.1 1 length=101
2	염기 서열	GTTAGCGTTGTTGATCGCG...
3	• 줄	+SRR1553607.1 1 length=101
4	품질 문자열 (서열과 길이가 같음)	BB@FFFFFHHHHHJJJJJ...

품질 문자는 **Phred+33** 방식으로 적혀 있다. 문자의 ASCII 번호에서 33을 빼면 Phred 점수가 된다.

문자	ord(ch)	점수 ord(ch) - 33
!	33	0
+	43	10
5	53	20
?	63	30
I	73	40
J	74	41

4. 출력 형식

한 줄에 위치 하나씩, **위치**와 **평균 품질 점수**를 탭(₩t)으로 구분하여 출력한다.

- 위치는 1부터 센다 (1번째 염기 = 1).
- 평균은 소수점 아래 둘째 자리까지 반올림한다.
- 줄 수는 가장 긴 read의 길이와 같다.

예:

```
1	32.04
2	32.17
3	32.28
```

5. 코드 전체

```
# FASTQ 파일의 모든 read에서 위치(1번째 염기, 2번째 염기, ...)별 평균 Phred 품질 점수를 구한다.
# 사용법: python3 src/qual_by_position.py <FASTQ 파일>
import sys

def read_fastq(filename):
    records = []
    with open(filename) as f:
        lines = f.readlines()
    for i in range(0, len(lines), 4):
        name = lines[i].rstrip()[1:]
        seq = lines[i + 1].rstrip()
        qual = lines[i + 3].rstrip()
        records.append((name, seq, qual))
    return records

reads = read_fastq(sys.argv[1])

# totals[i]: i번째 위치(0부터 셈)의 품질 점수 합계
# counts[i]: i번째 위치에 염기가 있는 read의 개수
totals = []
counts = []

for name, seq, qual in reads:             # 바깥 반복문: read 하나씩
    for i in range(len(qual)):            # 안쪽 반복문: 그 read의 위치 하나씩
        score = ord(qual[i]) - 33         # Phred+33 변환
        if i >= len(totals):              # 처음 만나는 위치이면 칸을 하나 늘린다
            totals.append(0)
            counts.append(0)
        totals[i] = totals[i] + score
        counts[i] = counts[i] + 1

for i in range(len(totals)):
    average = totals[i] / counts[i]
    print(str(i + 1) + "\t" + str(round(average, 2)))
```

6. 주요 코드의 동작 원리

6.1 FASTQ 읽기: read_fastq()

파일의 모든 줄을 읽은 뒤 4줄씩 끊어서 (이름, 서열, 품질) 튜플을 만든다. range(0, len(lines), 4)는 0, 4, 8, ... 처럼 4씩 건너뛰므로 i는 항상 각 read의 첫 줄(@ 줄)을 가리킨다. 품질 문자열은 그 read의 4번째 줄, 즉 lines[i + 3]이다. 이 함수는 같은 폴더의 qual_summary.py에 있는 것과 같다.

6.2 위치별로 모아 두는 두 리스트: totals, counts

- totals[i]: 위치 i의 품질 점수를 모두 더한 값
- counts[i]: 위치 i에 염기가 있었던 read의 개수

평균 = totals[i] / counts[i] 이다. 리스트 번호는 0부터 시작하므로 totals[0]이 1번째 염기, totals[100]이 101번째 염기에 해당한다.

6.3 반복문 안의 반복문

- **바깥 반복문** for name, seq, qual in reads: 는 read를 하나씩 꺼낸다.
- **안쪽 반복문** for i in range(len(qual)): 는 꺼낸 read의 품질 문자를 위치 0부터 끝까지 하나씩 본다.
- 안쪽에서 ord(qual[i]) - 33으로 점수를 구해 totals[i]에 더하고, counts[i]를 1 늘린다.

6.4 길이가 다른 read 처리

리스트는 처음에 비어 있다. 안쪽 반복문에서 i가 리스트 길이 이상이면(i >= len(totals)) 아직 그 위치의 칸이 없다는 뜻이므로 append(0)으로 칸을 하나 만든다. 그래서 미리 read 길이를 몰라도 되고, 가장 긴 read의 길이만큼 칸이 생긴다.

짧은 read는 자기 길이까지만 안쪽 반복문이 돌기 때문에, 그보다 뒤 위치의 counts는 늘리지 않는다. 즉 **그 위치에 염기를 가진 read만 평균에 들어간다.**

예를 들어 다음 두 read가 있으면

read	품질	점수
r1	IIII	40, 40, 40, 40
r2	!+	0, 10

위치	계산	평균
1	(40 + 0) / 2	20.0
2	(40 + 10) / 2	25.0
3	40 / 1	40.0
4	40 / 1	40.0

3, 4번 위치는 r1만 염기를 가지고 있으므로 r1 하나로 평균을 낸다.

6.5 출력

마지막 반복문은 위치마다 평균을 구해 str(i + 1) + "₩t" + str(round(average, 2)) 로 한 줄을 만든다. i + 1로 바꾸어 위치를 1부터 보여 준다.

7. 실제 실행 결과 (data/SRR1553607_1.fastq)

- read 개수: 203,445개, 모든 read의 길이는 101 (그래서 출력도 101줄)
- 실행 시간: 약 6.5초

```
1    32.04
2    32.17
3    32.28
4    35.85
5    35.65
...
48    33.92
49    33.72
50    33.44
...
99    29.03
100    28.93
101    28.25
```

앞쪽 위치(4~5번째)에서 약 35.8로 가장 높고, 뒤로 갈수록 점점 낮아져 마지막 101번째 위치는 28.25이다. 일루미나 read에서 흔히 보이는 "뒤쪽 품질 저하" 모습이다.

결과 검증

1. **awk로 따로 계산한 값과 비교**: 같은 파일을 awk로 위치별 평균을 따로 구해 101개 위치를 모두 비교했더니 차이가 0이었다.
2. **길이가 다른 read 테스트**: 6.4절의 r1, r2 두 read로 만든 작은 FASTQ를 넣었더니 20.0, 25.0, 40.0, 40.0이 나와 손으로 계산한 값과 같았다.

8. 주의할 점

- readlines()로 파일 전체를 읽으므로 큰 파일에서는 메모리 사용량이 많아질 수 있다.
- 각 read가 4줄로 구성된 FASTQ 파일만 처리할 수 있다.
- Phred+33 인코딩을 가정하므로 Phred+64 파일에서는 품질 점수가 잘못 계산된다.